

RayLib Curso parte prática

Guia de Execução Prática para Instrutores — Minicurso Raylib em C

Este roteiro serve como passo a passo para o instrutor conduzir live coding na lousa/projetor enquanto os alunos acompanham no computador.

Passo 1: Criar o Projeto Básico

Objetivo: Apresentar a estrutura mínima de um jogo na Raylib e validar se o ambiente de todos os alunos está compilando.

- **O que explicar:**
 - `#include "raylib.h"` traz as declarações da biblioteca.
 - `InitWindow(largura, altura, titulo)` aloca a janela no sistema operacional.
 - `SetTargetFPS(60)` define uma trava de taxa de quadros (evita o uso de 100% da CPU/GPU).
 - O loop `while (!WindowShouldClose())` roda continuamente até que o usuário clique no **X** ou aperte **ESC**.
 - `BeginDrawing()` e `EndDrawing()` delimitam o frame de renderização (pipeline gráfico).
 - `ClearBackground(RAYWHITE)` limpa o frame anterior para evitar artefatos de "rastro".

```
#include "raylib.h"

int main(void) {
    const int larguraTela = 800;
    const int alturaTela = 600;

    InitWindow(larguraTela, alturaTela, "Minicurso Raylib - Coletor");
    SetTargetFPS(60);

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);
        EndDrawing();
    }

    CloseWindow();
    return 0;
}
```

Passo 2: Desenhar um Círculo (E Responsividade)

Objetivo: Introduzir os conceitos de renderização de formas geométricas básicas e o uso do sistema de coordenadas da tela.

- **O que explicar:**
 - As funções `GetScreenWidth()` e `GetScreenHeight()` retornam as dimensões atuais da janela.
 - Usar `GetScreenWidth() / 2.0f` centraliza elementos dinamicamente, sem depender de valores fixos codificados em *hardcode*.
 - Apresentar a `struct Vector2` nativa da Raylib: `{ float x; float y; }`.
 - Apresentar a função `DrawCircleV(Vector2 center, float radius, Color color)`.

```
// --- ANTES DO GAME LOOP ---
Vector2 posJogador = { GetScreenWidth() / 2.0f, GetScreenHeight() / 2.0f };
float raioJogador = 20.0f;

// --- DENTRO DE BeginDrawing() / EndDrawing() ---
DrawCircleV(posJogador, raioJogador, BLUE);
```

Passo 3: Mover o Círculo com Teclado e Delta Time

Objetivo: Ensinar a captura de inputs contínuos do usuário e introduzir o conceito fundamental de **Delta Time**.

- **O que explicar:**
 - `IsKeyDown(KEY_RIGHT)` verifica se a tecla está pressionada no frame atual.
 - **Delta Time (`GetFrameTime()`):** É o tempo em segundos gasto para renderizar o último frame (ex: 0.00).
 - Multiplicar a velocidade pelo Delta Time transforma a taxa de movimento de **pixels por frame** para **pixels por segundo**, garantindo a mesma velocidade em qualquer taxa de atualização de monitor.

```
// --- ANTES DO GAME LOOP ---
float velocidade = 300.0f; // 300 pixels por segundo

// --- DENTRO DO GAME LOOP (ANTES DE BeginDrawing) ---
float dt = GetFrameTime(); // Delta Time

if (IsKeyDown(KEY_RIGHT) || IsKeyDown(KEY_D)) posJogador.x += velocidade * dt;
if (IsKeyDown(KEY_LEFT) || IsKeyDown(KEY_A)) posJogador.x -= velocidade * dt;
```

```
if (IsKeyDown(KEY_DOWN) || IsKeyDown(KEY_S)) posJogador.y += velocidade * dt;
if (IsKeyDown(KEY_UP) || IsKeyDown(KEY_W)) posJogador.y -= velocidade * dt;
```

Passo 3.1: Limitar o Personagem dentro das Bordas da Tela

Objetivo: Aplicar uma lógica simples de tratamento de limites numéricos (*clamping*).

- **O que explicar:**
 - Testes de limites para impedir que o centro do círculo ultrapasse a largura e a altura da janela.

```
// DENTRO DO GAME LOOP (Após processar o movimento do jogador)
if (posJogador.x - raioJogador < 0) posJogador.x = raioJogador;
if (posJogador.x + raioJogador > GetScreenWidth()) posJogador.x =
GetScreenWidth() - raioJogador;
if (posJogador.y - raioJogador < 0) posJogador.y = raioJogador;
if (posJogador.y + raioJogador > GetScreenHeight()) posJogador.y =
GetScreenHeight() - raioJogador;
```

Passo 4: Adicionar o Quadrado (Posição Aleatória)

Objetivo: Usar `structs` da Raylib para definir objetos com área e utilizar a geração de números aleatórios nativa.

- **O que explicar:**
 - Apresentar a `struct Rectangle: { float x; float y; float width; float height; }`.
 - Em vez de usar `rand()` da `<stdlib.h>`, usa-se `GetRandomValue(min, max)` da Raylib (já retorna inteiros dentro do intervalo).
 - `DrawRectangleRec(Rectangle rec, Color color)` desenha o retângulo diretamente a partir da `struct`.

```
// --- ANTES DO GAME LOOP ---
Rectangle item = {
    (float)GetRandomValue(50, larguraTela - 50),
    (float)GetRandomValue(50, alturaTela - 50),
    30.0f, 30.0f
};

// --- DENTRO DE BeginDrawing() / EndDrawing() ---
DrawRectangleRec(item, RED);
```

Passo 5: Detecção de Colisão

Objetivo: Ensinar lógica de colisão no espaço 2D utilizando funções utilitárias da Raylib.

- **O que explicar:**
 - O conceito de **Bounding Boxes / Hitboxes**.
 - A função `CheckCollisionCircleRec(Vector2 center, float radius, Rectangle rec)` executa o teste matemático entre a circunferência e o retângulo, retornando um valor booleano (`true` / `false`).
 - Quando a colisão ocorre, reposicionamos o item para uma nova coordenada aleatória.

```
// --- DENTRO DO GAME LOOP (NA SEÇÃO DE ATUALIZAÇÃO) ---
if (CheckCollisionCircleRec(posJogador, raioJogador, item)) {
    // Reposiciona o quadrado
    item.x = (float)GetRandomValue(50, GetScreenWidth() - 50);
    item.y = (float)GetRandomValue(50, GetScreenHeight() - 50);
}
```

Passo 6: Sistema de Pontuação e Exibição de Texto

Objetivo: Implementar estado do jogo (pontuação) e renderização de textos e variáveis numéricas na interface.

- **O que explicar:**
 - Atualização da variável de pontuação durante a colisão.
 - A função `TextFormat("Texto %d", variavel)` funciona como o `sprintf()`, formatando strings diretamente no momento da renderização.
 - A função `DrawText(const char *text, int posX, int posY, int fontSize, Color color)`.

```
// --- ANTES DO GAME LOOP ---
int pontuacao = 0;

// --- DENTRO DA CHECAGEM DE COLISÃO ---
if (CheckCollisionCircleRec(posJogador, raioJogador, item)) {
    pontuacao += 10; // Incrementa pontuação
    item.x = (float)GetRandomValue(50, GetScreenWidth() - 50);
    item.y = (float)GetRandomValue(50, GetScreenHeight() - 50);
}

// --- DENTRO DE BeginDrawing() / EndDrawing() ---
DrawText(TextFormat("Pontos: %d", pontuacao), 10, 10, 20, DARKGRAY);
```

Módulos Extras (Caso Sobre Tempo)

Se a turma evoluir rápido e restar tempo, utilize os tópicos extras abaixo.

Extra 1: Carregar e Desenhar Texturas (Imagens)

Objetivo: Ensinar o gerenciamento de ativos externos (substituindo formas vetoriais por artes em PNG).

- **O que explicar:**

- A estrutura `Texture2D` armazena imagens carregadas na memória VRAM da GPU.
- `LoadTexture("caminho/imagem.png")` abre a imagem do disco.
- **Atenção:** Toda textura alocada precisa ser desalocada ao final da execução com `UnloadTexture(texture)`.
- `DrawTextureV(texture, position, tint)` desenha a textura usando uma `Vector2`.

```
// 1. ANTES DO GAME LOOP (Carregamento)
Texture2D jogadorTex = LoadTexture("resources/player.png");
Texture2D itemTex = LoadTexture("resources/coin.png");

// 2. DENTRO DE BeginDrawing() / EndDrawing() (Substituir os DrawCircle e DrawRectangle)
// Nota: Para centralizar a textura na posição da colisão, subtrai-se a metade da largura/altura
DrawTexture(jogadorTex, posJogador.x - jogadorTex.width/2.0f, posJogador.y - jogadorTex.height/2.0f, WHITE);
DrawTexture(itemTex, item.x, item.y, WHITE);

// 3. APÓS O GAME LOOP (Desalocação)
UnloadTexture(jogadorTex);
UnloadTexture(itemTex);
```

Extra 2: Tocar Efeitos Sonoros (Áudio)

Objetivo: Introduzir o módulo de áudio da Raylib (`raudio`).

- **O que explicar:**

- É necessário inicializar a placa de som com `InitAudioDevice()` e fechá-la com `CloseAudioDevice()`.
- `LoadSound("caminho/som.wav")` carrega efeitos sonoros curtos na RAM.
- `PlaySound(som)` executa o áudio.

```
// 1. INICIALIZAÇÃO DO ÁUDIO
InitAudioDevice();

Sound somColeta = LoadSound("resources/coleta.wav");

// 2. NA DETECÇÃO DE COLISÃO
```

```
if (CheckCollisionCircleRec(posJogador, raioJogador, item)) {  
    PlaySound(somColeta); // Toca o efeito sonoro  
    pontuacao += 10;  
    item.x = (float)GetRandomValue(50, GetScreenWidth() - 50);  
    item.y = (float)GetRandomValue(50, GetScreenHeight() - 50);  
}
```

```
// 3. ENCERRAMENTO  
UnloadSound(somColeta);  
CloseAudioDevice();
```